Adding contract testing to your CI/CD pipelines requires a few configurations, especially if you want to trigger the provider job automatically every time there are contract changes published by the consumer.

It’s recommended to continuously improve this process and introduce the concepts of webhooks, versioning, and environments incrementally, which we’ll cover in more detail as part of chapter 10.

Summary

- Contract testing requires two sets of tests: one from the consumer and one from the provider. Without either of them, contract testing won’t give you any real benefits.
- Contract testing combines the best qualities of unit testing and integration testing.
- Contract testing doesn’t replace integration or E2E tests because it doesn’t verify any side effects on the system’s overall behavior.
- The red-green-refactor life cycle of TDD can also be likened to a contract testing life cycle.
- Contract testing improves the communication process between different teams by enforcing that any contract changes are verified before deploying to production.
- To get buy-in from the team, start with communicating why contract testing is needed.
- Start small by converting one integration test to a contract test and then continuously improve.
- Communicate the benefits and progress of your contract testing journey.
- Contract testing requires a team effort, so find allies or champions to help you.
- While there are many benefits, contract testing takes time and effort, so it might be challenging to implement it without the proper education.
- Contract testing is a highly technical activity and requires an understanding of advanced concepts not usually present in integration or E2E testing.